

A Practical Approach To Verifying Structural Invariants In Colored Petri Nets

Structural analysis is a core method in Petri Net (PN) research, offering a perspective complementary to state-space techniques while avoiding many of their limitations. It is well studied for classical PNs but far less for High-Level Petri Nets (HLPN). Symmetric Nets (SN), a common HLPN formalism, use compact annotations to encode behavioral symmetries, enabling the construction of a symbolic reachability graph (and a lumped Markov chain in stochastic SN) and the execution of symbolic discrete-event simulations.

Over the past two decades, structural techniques tailored to SN have been developed, notably supported by the SNexpression tool. This tool implements a formal calculus designed for the computation of symbolic structural relations, including, but not limited to, conflict relations and causal dependencies. Here, we focus on using this calculus to verify semi-automatically symbolic structural invariants, a task currently feasible only for certain restricted SN subclasses. We focus specifically on (semi)flows and briefly discuss an approach through which a flow generative family can be generated, at least theoretically. We further briefly outline a framework for the formal verification of a broader class of invariant properties. An extended formulation of the SN formalism, which demonstrably satisfies the closure property with respect to fundamental functional operators, is employed. Core concepts are elucidated by means of representative examples throughout the exposition.

1 Introduction And Related Work

High-Level Petri Nets (HLPNs) [17] constitute a broad family of Petri Net-based formalisms (including, among others, Algebraic Petri Nets, Predicate/Transition Nets, and Colored Petri Nets) that extend basic Petri Nets (PN) [18] by admitting structured, data-rich tokens in place of indistinguishable ones. In HLPNs, both places and transitions—the nodes of the underlying bipartite graph—are typed or parameterized, so that each node may admit multiple distinct instantiations. The arcs of an HLPN are labeled with expressions, which, depending on the specific formalism, can be algebraic terms, functions, or other forms of symbolic annotations. By enabling such data and type structures, HLPNs provide a more abstract and compact representation of complex concurrent systems.

In Colored Petri Nets (CPNs), in accordance with the original definition by Jensen [15, 16], the nodes of the net are associated with *finite color domains*. Each arc is annotated with a function from the color domain of the incident transition into multisets over the color domain of the incident place. A Colored Petri Net can be *unfolded* into an equivalent Place/Transition (P/T) net [15, 18]. In realistic scenarios, such an unfolding procedure may become excessively complex or even computationally intractable, and the subsequent back-interpretation of the analysis results may be rendered considerably more difficult.

Symmetric Nets (SN) [10], formerly referred to as Well-formed Nets, constitute a class of Coloured Petri Nets (CPNs) characterized by a dedicated syntactic structure explicitly tailored to exploit behavioral symmetries of the underlying model during its analysis. The domains associated with SN nodes are Cartesian products of basic *color classes*. A color class can be partitioned into static subclasses (or ordered circularly); colors in a subclass represent entities with similar behavior. SN functions encode the color domain structure of the nodes; arcs are annotated with expressions that are composed of tuples of

base color functions. Despite this syntactic specialization, SNs have been formally shown to be equally expressive as the original CPN formalism [15], in the strict sense that any CPN can be systematically transformed into an equivalent SN. Using a symbolic initial marking for an SN, together with a symbolic firing rule, one builds a compact Symbolic Reachability Graph, which is strongly bisimilar to the ordinary Reachability Graph. For Stochastic SN, this graph yields a lumped Markov chain; alternatively, it is possible to conduct simulations using a symbolic discrete-event simulation framework [1].

A distinctive advantage of Petri Nets is that one can obtain meaningful properties directly from their structure, without constructing the state space; extending this capability efficiently to CPNs, without using unfolding (that is, symbolically), is far from straightforward. The structural analysis of SN has seen substantial progress over the past two decades. The theoretical foundations are rooted in [3], which defines a language to symbolically express and calculate the principal structural relations (conflict, causal connection, mutual exclusion) [4]. The elements of this language are similar to the SN arc functions but show expanded expressivity by using guards also as function prefixes. The SNexpression software tool [5] (www.di.unito.it/~depierro/SNexpression), featuring a command-line interface (CLI) built on top of an extensible Java library, functions as a computer algebra system that reduces user-specified arbitrary structural expressions over pairs of SN nodes to comprehensible normal forms. In a recent version of the tool [7], this calculus has been extended to operate at the matrix level (i.e., over the entire SN), thereby facilitating the computation of more general structural dependencies, including the handling of priorities.

In this paper, we examine the effectiveness of the algebraic calculus underlying the SNexpression tool for the symbolic verification of structural *invariants* of an *extension* of SN (denoted ESN) that exhibits advantageous algebraic properties, with particular emphasis on (semi-)flows. Specifically, we exploit the fact that the ESN arc functions constitute a class that is closed under fundamental functional operators, most notably composition [6] and transposition. To the best of our knowledge, no alternative methodology has been proposed for the verification and computation of symbolic invariants without imposing constraints on SN color functions. In contrast, our approach is grounded in an extension of the SN formalism. The main goal is to show that the symbolic computation engine implemented in SNexpression efficiently supports semi-automatic formal verification of structural invariant properties, complementing the computation of structural dependencies.

In particular, we treat four distinct yet related topics. We first address the verification of symbolic (semi)flows, which currently cannot be directly generated or verified for general SN models, but only for certain restricted subclasses. We subsequently introduce a variant, referred to as colored flows, which abstracts away from quantitative characteristics (i.e., token multiplicities) and concentrates solely on qualitative properties (i.e., color types); in contrast to general symbolic flows, colored flows do not rely on conservative assumptions. Furthermore, we discuss how, in principle, one could derive a generative family (basis) of semiflows by exploiting the constant-size property of ESN functions and by relating symbolic semiflows to conventional semiflows of the underlying skeleton (a P/T net). Finally, we outline a more general approach based on standard inductive inference techniques, whose full automation, however, necessitates extensions of the supporting tool. Although the exposition adopts a formal style, all concepts are elucidated by means of illustrative examples.

Related work The computation of generative families of flows—especially positive flows—for classical Petri nets (PNs), i.e., integer solutions of a homogeneous linear system whose coefficient matrix is the net’s incidence matrix, has been extensively studied and effectively solved since [11]. Extending these techniques to High-Level Petri Nets (HLPN), in particular the automated verification and, when possible, computation of generative families of symbolic flows, has also been widely investigated. However, significant results exist only for restricted HLPN subclasses, mostly based on symbolic variants of clas-

sical elimination procedures (e.g., Gaussian or Gauss–Jordan elimination). For instance, [13] gives an algorithm for generative families of flows in Commutative High Level Nets, a subclass of Colored Petri Nets (CPN) where color functions form a ring of commutative endomorphisms. This method was later extended to Unary Regular Nets and unary Predicate/Transition nets in [12]. The method proposed in [19], which relies on generalized inverses of (linearly extended) arc functions, is in principle applicable to general CPNs. However, it has never been automated due to the practical infeasibility of computing these generalized inverses in symbolic form. Since then, only a few substantial theoretical advances have appeared; among them, [14] proposes a block-matrix–based algorithm for a subclass of Symmetric Nets (SN), namely Simple Well-formed Nets. Strict limitations on color functions (notably the lack of guards) make this technique unusable in many situations, including most examples in this paper.

The balance of the paper: In section 2, we present a formal description of the ESN syntax, preceded by a brief introductory explanation. Section 3 establishes some fundamental properties of the ESN color functions language. In section 4, we introduce symbolic (semi)flows and present some basic properties. The central section 5 illustrates (semi)flow verification through a series of examples that demonstrate the key features of the ESN syntax. In section 6, we briefly discuss the calculation of a semiflow basis for ESN. In section 7, we also define a class of colored flows where the multiplicities of colors are ignored. Section 8 briefly addresses the verification of more general structural invariants for ESN. We conclude by summarizing our contributions and indicating directions for ongoing work.

2 (Extended) Symmetric Nets

In this section, we present an extension of Symmetric Nets (SN), which we refer to as ESN. Throughout the remainder of this text, we work with the (E)SN formalism in the absence of stochastic parameters. We assume that the reader is already acquainted with the fundamental concepts of PN and HLPN; nevertheless, we provide a formal definition, preceded by a brief informal explanation to highlight the main syntactic components and the dynamics of the SN formalism.

2.1 (E)SN main components and dynamics

The domains of (E)SN nodes are Cartesian products of finite *color classes*. Each color class may be partitioned into static subclasses or ordered circularly. (E)SN arcs are annotated with (sums of) tuples of base color functions, such as projections and constants producing static subclasses or entire classes.

Consider the SN depicted in Fig. 2, while temporarily disregarding its semantic interpretation. The net employs two color classes, denoted by L and PL , which are neither ordered nor partitioned. The transition *work* is linked to the places ‘line’, ‘ass’, and ‘ok’ via an input arc, an output arc, and a combined input/output arc, respectively. Each of these places is assigned the color domain $PL \times L$. Each arc linking them to *work* is annotated with the function tuple $\langle pl, l \rangle$, where pl and l denote projection functions that can, at this point, be naturally interpreted as variables of types PL and L , respectively. Consequently, the implicit domain of the transition *work* is also $PL \times L$ (where the choice of the class disposition is arbitrary); The transition *work* is enabled to fire with an instance (or binding) $\langle pl = c, l = c' \rangle \in PL \times L$ if and only if both places ‘line’ and ‘ok’ each contain at least one occurrence of the token $\langle c, c' \rangle$. The firing of this instance removes one token (tuple) $\langle c, c' \rangle$ from place ‘line’ and produces one token of the same form in place ‘ass’.

Consider now the transition *load* from the same model. This transition is linked to the place ‘line’ by an output arc labeled with the function $\langle pl, All \rangle$, and to the place ‘start’, with the associated color domain

L , by an input arc labeled with the function $\langle All \rangle$; here, All denotes a constant function whose value is the entire color class L . Consequently, the (implicit) color domain of $load$ is PL . Observe that the symbol pl does not appear in any input arc function; consequently, it can be regarded as a free variable. An instance $\langle pl = c \rangle \in PL$ of $load$ is enabled if and only if every color in L is present at least once in ‘start’. Upon firing, this instance removes one occurrence of each color in L from ‘start’ and produces the multiset of tokens $\sum_{c' \in L} \langle c, c' \rangle$ in place ‘line’. Since variable pl is free, if the instance $\langle pl = c \rangle$ is enabled, then every other instance $\langle pl = c' \rangle$, $c' \in PL$, is also enabled, and all these instances are mutually in conflict (see the SN unfolding in the appendix, Fig. 13, for $|PL| = 2$ and $|L| = 2$, where we have two instances of $load$).

All remaining components of the SN formalism—namely, partitioned/ordered classes, guards, and functions expressed as algebraic sums—are illustrated in the examples presented in section 5. A distinguished ESN arc function (not admitted in the SN syntax) is presented at the end of section 2.3.

2.2 Multiset functions and operations

We deal with functions whose codomain consists of multisets and with functional operations. Let C be a non-empty set. A (generalized) multiset in C is a map $\mathbf{a} : C \rightarrow \mathbb{Z}$, where $\mathbf{a}(c)$ is the multiplicity of c . $|\mathbf{a}| := \sum_{c \in C} \mathbf{a}(c)$. We denote by $Bag^*[C]$ the set of generalized multisets in C and by $Bag[C]$ ($\subset Bag^*[C]$) the set $\{C \rightarrow \mathbb{N}\}$. $\mathbf{0} \in Bag[C]$ has zero multiplicities. The support of $\mathbf{b} \in Bag[C]$, denoted $\bar{\mathbf{b}}$, is $\{c \in C \mid \mathbf{b}(c) > 0\}$. Basic operations generalize in a natural way.

Let $\mathbf{a}, \mathbf{b} \in Bag^*[C]$, $\lambda \in \mathbb{Z}$, $\mu : \mathbb{Z}, \mathbb{Z} \rightarrow \mathbb{Z}$. $\lambda \mathbf{a} \in Bag^*[C] := \lambda * \mathbf{a}(c)$ (scalar product); $\mathbf{a} \mu \mathbf{b} \in Bag^*[C] := \mathbf{a}(c) \mu \mathbf{b}(c)$, $\forall c \in C$. Let $\rho : \mathbb{Z}, \mathbb{Z} \rightarrow Bool$. Then $\mathbf{a} \rho \mathbf{b} := \bigwedge_{c \in C} \mathbf{a}(c) \rho \mathbf{b}(c)$. Some operations (in $\mathbb{N}$) only apply to $Bag[C]$: in particular, $\min_{\mathbb{N}, \mathbb{N}}$ (also denoted by $\cap$) and $\ominus_{\mathbb{N}, \mathbb{N}}$ ($n_1 \ominus n_2 := \min(n_1 - n_2, 0)$).

Let $\mathbf{a}' \in Bag^*[C']$. $\mathbf{a} \times \mathbf{a}' \in Bag^*[C \times C'] := \mathbf{a}(c) * \mathbf{a}'(c')$, $\forall (c, c') \in C \times C'$ (Cartesian product). From now on, we will use the tuple notation $\langle \mathbf{a}, \mathbf{a}' \rangle$ in place of $\mathbf{a} \times \mathbf{a}'$.

The operations on multisets retain the properties of the underlying ones. Associative operations (including $\langle _, _ \rangle$) extend naturally to n -ary operations.

Functional operators Multiset operations can be extended in a functional manner via function evaluation. For example, let $f, h \in \{A \rightarrow Bag^*[C]\}$, $\lambda \in \mathbb{Z}$, $f' \in \{A \rightarrow Bag^*[C']\}$:

$\lambda f \in Bag^*[C] := \lambda f(a)$; $f + h \in \{A \rightarrow Bag^*[C]\} := f(a) + h(a)$; $\langle f, f' \rangle \in \{A \rightarrow Bag^*[C \times C']\} := \langle f(a), f'(a) \rangle$, $\forall a \in A$; $f < h := \bigwedge_{a \in A} f(a) < h(a)$. Some operators are applied to functions with codomain $Bag[C]$: these include $\cap, \ominus$ and functional support $\bar{f}$. A peculiar and fundamental functional operation is *transposing*. Let $f \in \{A \rightarrow Bag^*[C]\}$. Then $f^t \in \{C \rightarrow Bag^*[A]\} := f(a)(c)$, $\forall c \in C, a \in A$.

Linear extension and guards Multiset functions are linearly extended to take multisets as arguments. The linear extension of $f \in \{A \rightarrow Bag^*[B]\}$ is $f^e \in \{Bag^*[A] \rightarrow Bag^*[B]\} := \sum_{a \in A} \mathbf{a}(a) f(a)$, $\forall \mathbf{a} \in Bag^*[A]$. We shall use the same symbol for a function and its linear extension, applying the latter to multisets. Thus, we can simply define the composition. Let $g \in \{B \rightarrow Bag^*[C]\}$. Then $g \circ f \in \{A \rightarrow Bag^*[C]\} := g(f(a))$, $\forall a \in A$.

We deal with multiset functions that can have a guard as a suffix or prefix. Let $p \in \{A \rightarrow Bool\}$. Then $[p] \in \{A \rightarrow Bag^*[A]\} := a$ if $p(a)$; otherwise $\mathbf{0}$, $\forall a \in A$. Let $f \in \{A \rightarrow Bag^*[B]\}$ and $p' \in \{B \rightarrow Bool\}$. Then $f[p] := f \circ [p]$ and $[p']f := [p'] \circ f$. A guard used as a prefix is called a *filter*.

2.3 Extended SN

Definition 2.1 (Extended SN). *An ESN is a tuple:*

$$\mathcal{N} = (P, T, \Sigma, \mathcal{D}, I, O, H, \Phi, \mathbf{m}_0)$$

where P and T are finite, nonempty, disjoint sets holding the places and the transitions; Σ is the set of color classes; $(\neq \emptyset)$; $\mathcal{D}$ assigns a color domain to each $v \in P \cup T$; I, O and H are families of arc functions defined for each $(p, t) \in P \times T$; Φ assigns each $t \in T$ a guard; a marking $\mathbf{m}$ is a P -vector such that $\mathbf{m}(p) \in \text{Bag}[\mathcal{D}(p)]$; $\mathbf{m}_0$ denotes the initial marking.

- The color classes in $\Sigma = \{C_i, i = 1 \dots n\}$ are finite pairwise disjoint sets of elements called *colors*; each class C_i can be *partitioned* into static subclasses $\{C_{i,j}\}_{j:1 \dots s_i}$ or alternately *ordered* circularly; if C_i is ordered, then $!^m(c)$ ($m \in \mathbb{Z}$) denotes the successor/predecessor m^{th} of $c \in C_i \pmod{|C_i|}$.
- A color domain $\mathcal{D}_\Sigma$ is a Cartesian product $\bigotimes_{i=1}^n C_i^{e_i}$, where $e_i \in \mathbb{N}$ denotes the repetitions of class C_i (C_i^0 is the identity); $|\mathcal{D}_\Sigma| := \sum_{i=1}^n e_i$. We define a partial order in the color domains: $\mathcal{D}_\Sigma \leq \mathcal{D}'_\Sigma$ if and only if $\forall i, e_i \leq e'_i$.
- The color domain $\mathcal{D}(p)$ of a place p defines the shape of tokens (color tuples) that can exist in p ; the marking of p is a multiset in $\mathcal{D}(p)$;
- The color domain $\mathcal{D}(t)$ of a transition t defines the instances of t ; $\mathcal{D}(t)$ can be inferred from the inscriptions of the arcs surrounding t ;
- The guard $\Phi(t)$ of a transition t is a function $\mathcal{D}(t) \rightarrow \text{Bool}$ syntactically expressed through a *standard predicate* (defined below); the color domain of t is *restricted* to $\{c \in \mathcal{D}(t) \mid \Phi(t) = \text{true}\}$. For simplicity, the symbol $\mathcal{D}(t)$ will denote such a restricted domain; $c \in \mathcal{D}(t)$ will also be denoted (t, c) ;
- Let $W \in \{I, O, H\}$. An arc function $W(p, t) \in \{\mathcal{D}(t) \rightarrow \text{Bag}[\mathcal{D}(p)]\}$ belongs to the language $\mathcal{L}_{\mathcal{D}(t), \mathcal{D}(p)}$ that is formally defined as:

$$\mathcal{L}_{\mathcal{D}_\Sigma, \mathcal{D}'_\Sigma} := \{F := \sum_i \lambda_i \cdot [g'_i] T_i [g_i], \lambda_i \in \mathbb{Z}\} \cup \{0_{\mathcal{D}_\Sigma, \mathcal{D}'_\Sigma}\} \quad (1)$$

T_i is a *function tuple* $\langle f_1, \dots, f_{|\mathcal{D}'_\Sigma|} \rangle$, and g_i and g'_i are *standard predicates* defined in $\mathcal{D}_\Sigma$ and $\mathcal{D}'_\Sigma$, respectively. Each f_l in T_i , called a *class function*, is a map $\mathcal{D}_\Sigma \rightarrow \text{Bag}^*[C_h]$, where $C_h \in \Sigma$ occurs at the position l in $\mathcal{D}'_\Sigma$. For a certain $\mathcal{D}_\Sigma$ and $C_h \in \Sigma$, a class function is expressed as

$$f \in \{\mathcal{D}_\Sigma \rightarrow \text{Bag}^*[C_h]\} := \sum_k \alpha_k \cdot \varepsilon_k, \alpha_k \in \mathbb{Z} \quad (2)$$

$\varepsilon_k \in \{\mathcal{D}_\Sigma \rightarrow \text{Bag}[C_h]\}$ can be: a projection x_h^j , $1 \leq j \leq e_h$ ($x_h^j(c)$ results in the j^{th} color of class C_h in the color tuple c), possibly preceded by $!^m$ if C_h is ordered; a constant function All_h , which maps to C_h (seen as a multiset); a constant function $All_{h,q}$, which maps to $C_{h,q}$, if C_h is partitioned. $g_i \in \{\mathcal{D}_\Sigma \rightarrow \text{Bool}\}$ is defined in terms of the basic clauses: $x_h^j = (\neq) x_h^y$, $x_h^j = (\neq)!^m x_h^y$ if C_h is ordered and $x_h^j \in C_{h,q}$ if C_h is partitioned.

Note that in (1) and (2) there can be negative terms, but arc-functions must always map to $\text{Bag}[\mathcal{D}(p)]$.

Semantics A transition instance (t, c) is *enabled* in the marking $\mathbf{m}$ if:

$$\forall p \in P, I(p, t)(c) \leq \mathbf{m}(p) \wedge (H(p, t)(c) = 0 \vee H(p, t)(c) > \mathbf{m}(p))$$

If (t, c) is *enabled* in $\mathbf{m}$, it can fire, leading to $\mathbf{m}' := \mathbf{m} + O(p, t)(c) - I(p, t)(c)$ (denoted $\mathbf{m}[(t, c) > \mathbf{m}']$).

Conventions In the following, we use capital words, e.g., C, L, PL , to denote basic color classes in Σ , and the corresponding lowercase words to denote projections (variables) of the respective class (with a subscript if that class is repeated in a color domain): for example, if $\mathcal{D}_\Sigma = L^2 \times PL$, then the projection symbols in that domain are $\{l_1, l_2, pl\}$.

ESN Arc function example Consider the following function, appearing in Figure 4, whose semantics will be elucidated in Section 5. This function cannot be represented within the original SN syntax.

$$\langle All - n, n, ack \rangle + [n_1 \neq n_2] \langle All - n, All - n, ack \rangle \in \{N \rightarrow N^2 \times L\}$$

This function occurs on the input arc from the place delivered, whose domain is $N^2 \times L$, to the transition *complete*, whose inferred domain is simply N . The color class L is partitioned into $L_1 := \{data\}$ and $L_2 := \{ack\}$; here, the symbol *ack* stands for All_{L_2} . The color class N consists of K colors (a model parameter) that are not distinguishable individually. For a given color $c \in N$, the function returns the (multi)set containing all triples $\langle c', c, ack \rangle$ for every $c' \neq c$ (corresponding to the first term), together with all triples $\langle c'', c''', ack \rangle$ for arbitrary c'', c''' such that $c'' \neq c$, $c''' \neq c$, and $c''' \neq c''$ (the second term). A central aspect is determining the size of the images of the arc functions, a nontrivial task due to possible filters, for which chromatic polynomials of graphs are used [6].

3 Base Properties Of Arc Functions And Equivalence Transformations

The symbol $\mathcal{L}$ will henceforth refer to the language of arc functions (1), whose (co-)domains are built over the set of color classes Σ associated with a given ESN. In this section, we outline the main algebraic properties of $\mathcal{L}$ and introduce basic structural transformations that support its symbolic manipulation.

Proposition 3.1. $\mathcal{L}$ is closed under the base functional operations; composition ($\circ$), transposition (t), algebraic sum ($+(-)$), difference ($\ominus$), and intersection ($\cap$).

The same property holds for the derived language $\overline{\mathcal{L}} := \{\overline{F} \mid F \in \mathcal{L}\}$, consisting of functions in $\{\mathcal{D}_\Sigma \rightarrow 2^{\mathcal{D}_\Sigma}\}$, where functional operators apply to sets. The base structural dependencies among the SN nodes can be expressed symbolically in $\overline{\mathcal{L}}$, whose syntax matches that of $\mathcal{L}$, except that class functions can use the operator $\cap$, the scalars are restricted to $\mathbb{N}$, and only one difference symbol ($\ominus$) is allowed.

It may be convenient to translate the SN annotations into "equivalent" forms that meet some requirements. In particular, (E)SN arc functions should be assumed to be of constant-size:

Definition 3.1. $F \in \{A \rightarrow Bag^*[B]\}$ is constant-size if $\exists k \in \mathbb{Z} \forall a, |F(a)| = k$.

Proposition 3.2. An SN transition t can be split into $t'_1, \dots, t'_k$ that adopt uniquely constant size arc functions, such that $\mathcal{D}(t) = \bigcup_{i=1}^k \mathcal{D}(t'_i)$, $\mathcal{D}(t'_i)$ are pair-wise disjoint, and $\mathbf{m}[(t, c) > \mathbf{m}']$ if and only if $\mathbf{m}[(t'_i, c) > \mathbf{m}']$ for some t'_i .

An example is presented in Fig. 1: The function $O(p_1, t)$ has a size that depends on the guard of one term; therefore, t is split (on the right) so that the functions satisfy the constant-size requirement.

We can also express any $W \in \mathcal{L}$ as $\sum_i \lambda_i F_i$, where the terms F_i are pair-wise disjoint and each F_i produces constant-size multisets in which all element multiplicities are equal to one. For example, after performing the transformation, we can readily confirm that the codomain of W is $Bag[B]$ by checking that all coefficients λ_i belong to $\mathbb{N}$.

A concluding observation regards the equivalence test. In general, expressions W in $\mathcal{L}$ do not admit a canonical representative, although their constituent components F_i do. However, for any $W, W' \in \mathcal{L}_{\mathcal{D}_\Sigma, \mathcal{D}_\Sigma'}$, the equivalence $W \equiv W'$ can be decided syntactically by examining $W - W'$ and $W' - W$. In fact, our symbolic calculus ensures that null expressions have a reduced syntactic form 0.

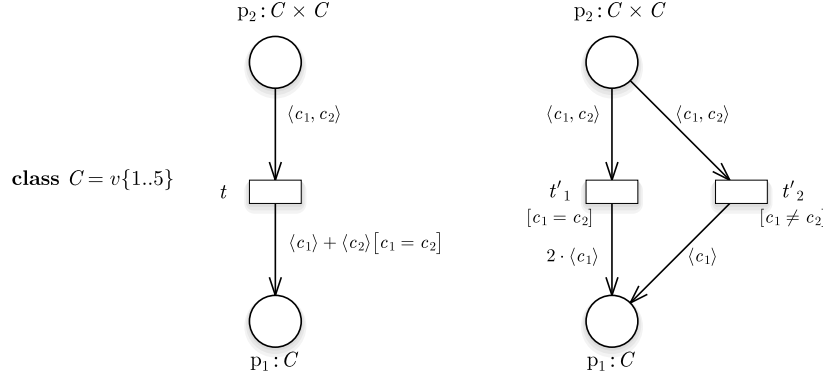


Figure 1: Transformation into constant-size functions

4 Symbolic (Semi)Flows: Basic Definitions And Properties

As stated above, hereinafter we implicitly consider constant-size functions. Let $\mathbf{H}$ be the $|P| \times |T|$ incidence matrix of an SN: $\mathbf{H}[p, t] = O(p, t) - I(p, t)$. The symbol ' $\cdot$ ' denotes the usual product of function matrices based on the composition of functions. Assume that places and transitions are indexed; we can use the notation $\mathbf{H}[i, j]$ instead of $\mathbf{H}[p_i, t_j]$

Definition 4.1 (P-(semi)flow). *Let $\mathbf{I}_P$ be a non-null $1 \times |P|$ vector of functions such that $\forall p, \mathbf{I}_P[p] \in \{\mathcal{D}(p) \rightarrow \text{Bag}^*[\mathcal{D}']\}$, and $\mathbf{I}_P[p] \neq 0 \Rightarrow \mathcal{D}' \leq \mathcal{D}(p)$.*

$\mathbf{I}_P$ is a P-flow (P-invariant) if and only if $\mathbf{I}_P \cdot \mathbf{H} \equiv \mathbf{0}$.

A P-flow such that $\forall p$, and $\mathbf{I}_P[p] \in \{\mathcal{D}(p) \rightarrow \text{Bag}[\mathcal{D}']\}$ is called P-semiflow.

A (semi)flow with minimal support (that is, which cannot be obtained as a linear combination of others) is said to be minimal. We expressly refer to minimal flows in the following.

Although general flows may reveal interesting properties, in our examples, we mostly focus on semiflows, which have an intuitive interpretation. If $\mathbf{m}_0$ is the initial SN marking and $\mathbf{I}_P$ is a semiflow, then for each reachable marking $\mathbf{m}$, $\mathbf{I}_P \cdot \mathbf{m} = \mathbf{I}_P \cdot \mathbf{m}_0$. From a computational point of view, there is no difference.

Analogously, T-flows are "right" solutions of a homogeneous system that has $\mathbf{H}$ as a symbolic coefficient matrix. Let us focus on semiflows.

Definition 4.2 (T-semiflow). *Let $\mathbf{I}_T$ be a non-null $|T| \times 1$ vector of functions such that $\mathbf{I}_T[t] \in \{\mathcal{D}' \rightarrow \text{Bag}[\mathcal{D}(t)]\}$ and $\mathbf{I}_T[t] \neq 0 \Rightarrow \mathcal{D}' \leq \mathcal{D}(t)$.*

$\mathbf{I}_T$ is a T-semiflow if and only if $\mathbf{H} \cdot \mathbf{I}_T \equiv \mathbf{0}$.

The T-semiflows also admit an intuitive interpretation. Let $\mathbf{I}_T$ be a T-semiflow, and let $\mathbf{A}$ belong to the image of $\mathbf{I}_T$ ($\mathbf{A}[t] \in \mathcal{D}(t)$); then, from each marking $\mathbf{m}$, any sequence of transition instances conforming to $\mathbf{A}$ leads back to $\mathbf{m}$.

These symbolic flows represent invariant properties of both quantitative and qualitative nature with respect to the entire ESN. By the way, it is also possible to limit the flow calculation to a specific subnet.

It should be noted that the above definitions of symbolic flows differ slightly from those commonly found in the literature (e.g., [14]). There, for instance, a common minimal super-domain for places is constructed, when needed, by inserting *All* constant functions into the function tuples to obtain a model that is equivalent with respect to bisimulation criteria.

4.1 Properties

Under the implicit assumption that all functions we consider have a constant size, an intuitive but pertinent property establishes a connection between symbolic and conventional flows.

Definition 4.3. *The skeleton of an ESN $\mathcal{N} = (P, T, I, O, \dots)$ is a PT net (P, T, F) . $F : P \times T \cup T \times P \rightarrow \mathbb{N}$ is such that $F(p, t) = \text{size}(I(p, t))$, $F(t, p) = \text{size}(O(p, t))$.*

Property 4.1. *Let $\mathcal{N}_{skel}$ be the skeleton of the ESN $\mathcal{N} = (P, T, \dots)$. If the $1 \times |P|$ vector $\mathbf{I}_P$ is a symbolic P-flow for $\mathcal{N}$ (Def. 4.1) then the derived vector in $\mathbb{Z}$ $\mathbf{I}'_P$ such that $\mathbf{I}'_P[p] = \text{size}(\mathbf{I}_P[p])$ is a conventional P-flow for $\mathcal{N}_{skel}$.*

An analogous connection exists between the symbolic T-flows of $\mathcal{N}$ and those of $\mathcal{N}_{skel}$. This property enables a substantial reduction of the search space for candidate flows. A potential application of this result to the determination of a generative basis of semiflows will be briefly addressed in Section 6.

Let $\mathbf{H}$ be the incidence matrix of $\mathcal{N}$. Its transpose $\mathbf{H}^t$ is a $|T| \times |P|$ matrix such that $\mathbf{H}^t[i, j] = \mathbf{H}[j, i]^t$ (this operation is different from the conventional matrix transposition). Analogously, $\mathbf{I}'_P$ is a vector $|P| \times |1|$ such that $\mathbf{I}'_P[j] = \mathbf{I}_P[j]^t$. The following property states that there exist two equivalent methodologies to validate candidate flows. This can be convenient from a computational perspective. An analogous property is valid for T-flows.

Property 4.2. *Let $\mathbf{I}_P$ be a non-null $1 \times |P|$ vector. Then*

$$\mathbf{I}_P \cdot \mathbf{H} \equiv \mathbf{0} \Leftrightarrow \mathbf{H}^t \cdot \mathbf{I}'_P \equiv \mathbf{0}$$

This property also sets a relationship between the P- and T-flows. A P-flow of $\mathcal{N}$ is a T-flow for the dual net $\mathcal{N}^d$, obtained by swapping places with transitions (reverting to the arc directions) and replacing the original arc functions with their transposes. And vice versa, a T-flow of $\mathcal{N}$ is a P-flow for $\mathcal{N}^d$.

5 Semi-Flows Verification: A Few Examples

In this section, we validate candidate P and T-semiflows using several examples. Each of these examples outlines specific aspects of the ESN syntax. Two are drawn from the literature, while the others have been constructed for illustrative purposes. The calculations were carried out semi-automatically with the support of SNexpression. In particular, the current implementation still does not natively support the composition in $\mathcal{L}$. Nevertheless, one can simulate this composition via that in $\bar{\mathcal{L}}$, provided the arc functions are (re)expressed as pairwise disjoint linear combinations $\sum_i \lambda_i F_i$ where terms F_i are guarded tuples that map to multisets in which all multiplicities are equal to one (which is the natural choice of modelers). The single calculation steps take negligible time. All models were edited using the GreatSPN package [1], which also supports the SN unfolding and the computation of conventional (numerical) semiflows. A translator from the GreatSPN format (<https://github.com/greatspn/SOURCES>) to the SNexpression format can be downloaded from the homepage.

Example 1 The first (Figure 2) refers to the core component of a distributed production system that gracefully degrades and is used as a reference in [8, 9]. The system has N parallel replicas of a production line (PL), each with K mutually interchangeable components subject to controlled degradation (we disregard these aspects here). The SN model employs two color classes L and PL , of sizes K and N , respectively (the latter are the model's parameters). The PT nets representing the unfolding of this SN (for $K = 2, N = 2$) and its skeleton are given in the appendix (Fig. 13, 14).

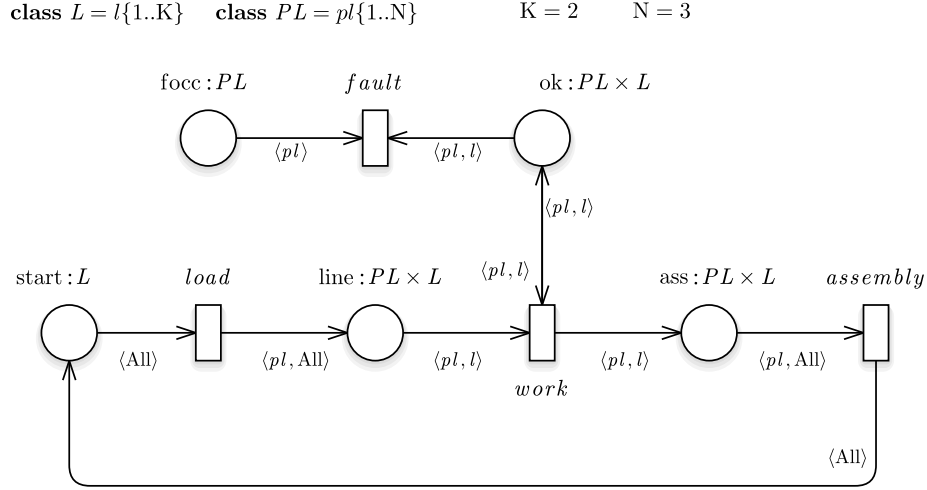


Figure 2: Distributed Production Line

	$load_{PL}$	$work_{PL,L}$	$assembly_{PL}$	$fault_{PL,L}$
start	$-\langle All_L \rangle$		$\langle All_L \rangle$	
line	$\langle pl, All_L \rangle$	$-\langle pl, l \rangle$		
ass		$\langle pl, l \rangle$	$-\langle pl, All_L \rangle$	
ok				$-\langle pl, l \rangle$
focc				$-\langle pl \rangle$

	start	line	ass	ok	focc
$\mathbf{I}_P^1$	$[\langle l \rangle_L]$	$\langle l \rangle_{PL,L}$	$\langle l \rangle_{PL,L}$		
$\mathbf{I}_P^2$				$-\langle pl \rangle_{PL,L}$	$\langle pl \rangle_{PL}$

	$load_{PL}$	$work_{PL,L}$	$assembly_{PL}$	$fault_{PL,L}$
$\mathbf{I}_T$	$\langle pl \rangle_{PL}$	$\langle pl, All_L \rangle_{PL}$	$\langle pl \rangle_{PL}$	

Figure 3: Incidence Matrix and P- and T-semiflows of SN in Fig. 2

Fig. 3 presents the incidence matrix together with the potential (semi)flows, as derived from property 4.1 (with the T-vector also displayed as a row), which can be simply verified using SNexpression CLI. The color domains of transitions, as well as the domains of functions, are explicitly shown. Observe that the same symbol (e.g., l) may denote projections over different domains.

Semiflow $\mathbf{I}_P^1$ admits a straightforward interpretation. It indicates that the distributed PLs (the lower part of the figure) are conservative with respect to the color class L : for example, if the initial marking contains a multiple of K distinct tokens (representing workpieces) in place start, this quantity is preserved throughout execution. $\mathbf{I}_P^2$ is the only instance of a flow that does not match a semiflow reported in the paper. This implies that the PL component associated with the place ok decreases consistently with the marking of the place focc. Because the SN places are covered by (semi)flows, the SN is structurally bounded (and color-safe, provided that the initial marking is color-safe).

Even more interestingly, the T-semiflow identifies a production cycle within a given PL, represented by the symbol pl : this cycle consists of an occurrence of *load*, K occurrences of *work*, and an occurrence

of *assembly*. Observe that these symbolic semiflows are intrinsically parametric, since no multiplicities appear either in $\mathbf{H}$ or in the semiflow expressions.

We now briefly analyze the relationship between symbolic and conventional (semi)flows, with reference to the unfolding of Fig. 2 reported in the Appendix (see Fig. 13). Consider first $\mathbf{I}_p^1$. We note that the components associated with the places *line* and *ass* correspond to the projection onto class L of the color domain $PL \times L$. Consequently, for a given color of L bound to l , there exist as many conventional semiflows as there are possible combinations of PL -colors in these two places. This yields a total of $N^2 \cdot K$ conventional semiflows that instantiate $\mathbf{I}_p^1$. For example, when $N = 2$ and $K = 2$, we obtain the eight P-semiflows shown in the Appendix (Fig. 13).

The symbolic T-semiflow, in turn, corresponds to N conventional semiflows, each associated with a distinct binding of pl (see Fig. 13).

Example 2 The second example in this section is a variant of one presented in [2], and it models a broadcast communication protocol. This model necessitates the utilization of the extended SN syntax. The ESN shown in Fig. 4 (and the following) uses more sophisticated functions. The class N denotes the nodes in a network, while the class L is divided into two singleton subclasses that encode *data* and *acknowledge* messages, respectively. The protocol operates as follows: when a message l is to be transmitted from a source node n (the place *sender*), it is sent to all other nodes in the network (transition *broadcast*). Once a *data* message has been delivered to a node, that node schedules an acknowledgment using the same broadcast mechanism. The protocol terminates successfully when all nodes, including the original sender, have received an acknowledgment.

A notable component of the SN is the function $I(\text{delivered}, \text{complete})$, previously described, which represents a sophisticated synchronization condition: all other nodes have given an acknowledgment to node n (the original sender), and each of these nodes has received an acknowledgment from all nodes except n and itself. Within this function, a filter is applied to extract all pairs of distinct nodes from the parametric Cartesian product $(All - n) \times (All - n)$.

The ESN skeleton is also shown in the Appendix. Since it does not allow any P-semiflows, the ESN consequently has no symbolic P-semiflows either. In contrast, it admits the following conventional T-semiflow (with $K = |N|$):

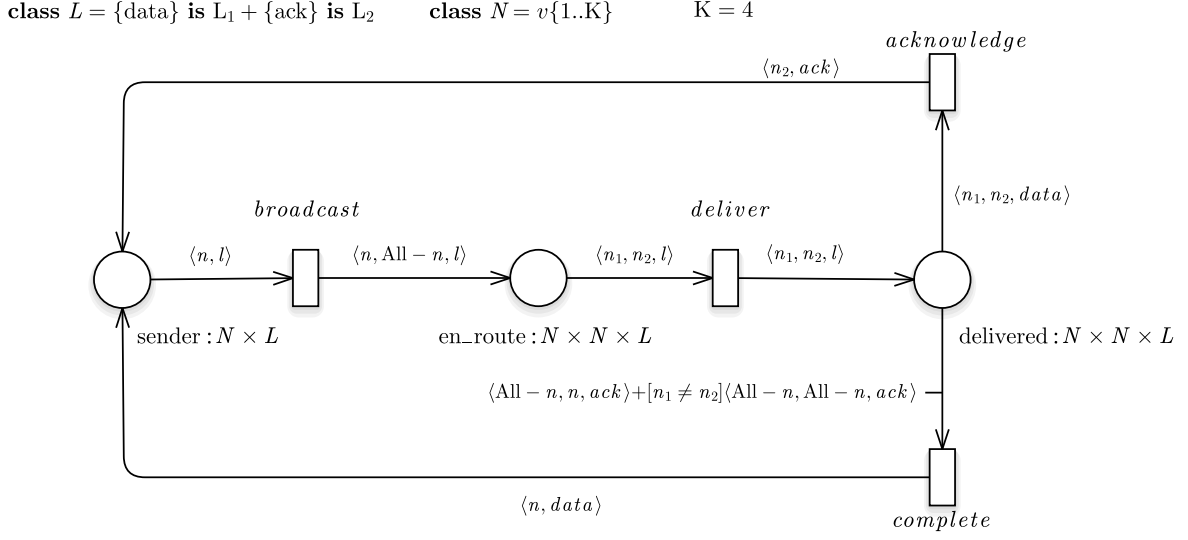
$$[\text{broadcast} : K, \text{deliver} : K \cdot (K - 1), \text{acknowledge} : K - 1, \text{complete} : 1].$$

The candidate symbolic semiflow shown in Fig. 5 is in agreement with this. To confirm that it is indeed a T-semiflow, we must resolve nontrivial compositions like the one below, which arises when multiplying the second row of $\mathbf{H}$ by $\mathbf{I}_T$. The symbolic calculus reduces all expressions e to a normal form $\hat{e} \in \mathcal{L}$.

The equivalence of terms can be established purely syntactically since the calculus guaranties that, whenever $e \equiv 0$, it follows that $\hat{e} = 0$.

$$\begin{aligned} & \langle n, All - n, l \rangle \circ \langle All - n, ack \rangle \equiv \\ & \langle n, All, l \rangle \circ \langle All - n, ack \rangle - \langle n, n, l \rangle \circ \langle All - n, ack \rangle \equiv \\ & \langle All - n, All, ack \rangle - [n_1 == n_2] \langle All - n, All - n, ack \rangle \equiv \\ & [n_1 \neq n_2] \langle All - n, All - n, ack \rangle + \langle All - n, n, ack \rangle \end{aligned}$$

We can directly verify that the row-by-column products are zero and therefore $\mathbf{I}_T$ is a semiflow. It corresponds to the completion of a basic protocol cycle: for any n representing the original sender, and

Figure 4: An ESN representing a broadcast protocol ($ack \equiv All_{L_2}$, $data \equiv All_{L_1}$)

$$\begin{array}{l}
 \text{sender} \\
 \text{en_route} \\
 \text{delivered}
 \end{array}
 \begin{array}{c}
 \begin{array}{c}
 broadcast_{N,L} \\
 deliver_{N,N,L} \\
 acknowledge_{N,N} \\
 complete_N
 \end{array}
 \end{array}
 \begin{bmatrix}
 -\langle n, l \rangle & & \langle n_2, ack \rangle & \langle n, data \rangle \\
 \langle n, All - n, l \rangle & -\langle n_1, n_2, l \rangle & & \\
 & \langle n_1, n_2, l \rangle & -\langle n_1, n_2, data \rangle & \\
 & & & -\langle All - n, n, ack \rangle \\
 & & & -[n_1 \neq n_2] \langle All - n, All - n, ack \rangle
 \end{bmatrix}$$

$$\mathbf{I}_T
 \begin{bmatrix}
 broadcast & deliver & acknowledge & complete \\
 (\langle n, data \rangle + \langle All - n, ack \rangle)_N & (\langle n, All - n, data \rangle + \langle All - n, n, ack \rangle + [n_1 \neq n_2] \langle All - n, All - n, ack \rangle)_N & \langle n, All - n \rangle_N & \langle n \rangle_N
 \end{bmatrix}$$

Figure 5: Incidence Matrix and T-semiflow of SN in Fig. 4

ignoring the specific order, it includes one *broadcast* of a *data* message from n , and *broadcasts* of an *ack* message from all the other nodes; a set of *deliver* events of the *data* message from n to every other node, a set of *deliver* events of an *ack* from each other node to n , and a set of *deliver* events of an *ack* from any node other than n to any other node distinct from both n and itself; the emission of an *acknowledge* by every node other than n ; and finally a *complete* event, which represents the synchronization upon reception of all expected *ack* messages by n and by all other nodes.

Example 3 The SN shown in Fig. 6 uses an ordered color class C and a color domain in which this class is replicated. The possible symbolic semiflows (Fig. 7) correspond to the semiflows of the underlying skeleton, namely $[p_1 : 1 \ p_2 : |C| \ p_3 : 1]$ and $[t_1 : 1 \ t_2 : 1 \ t_3 : |C| - 1]$. Using the algorithm defined in [6] verify, for example, that the product of $\mathbf{I}_p^1$ by the first column of $\mathbf{H}$ is null. We obtain:

$$\langle c_2 \rangle \circ \langle !c, c \rangle + N \langle c \rangle \circ - \langle c \rangle + \langle c_1 \rangle \circ \langle c, All - !c \rangle \equiv \langle c \rangle - N \langle c \rangle + (N - 1) \langle c \rangle$$

We can similarly prove that the other row-by-column products are nullified.

Regarding the interpretation, $\mathbf{I}_p^1$ expresses that for any reachable marking, the colors appearing in the second component of the 2-tuples in p_1 , together with those in the first component of the tuples in p_3 , always sum up to $N = |C|$ times the multiset of colors in p_2 . Dually, $\mathbf{I}_p^2$ can be read as follows: in all reachable markings $\mathbf{m}$, the colors in the first component of the 2 tuples in p_1 plus those in the second component of the tuples in p_3 always form the entire color set C , where each color occurs with multiplicity $|\mathbf{m}(p_2)|$. The T-semiflow, instead, asserts that every firing sequence consisting of one occurrence $\langle c \rangle$ of t_1 and t_2 , together with all occurrences $\langle c, c' \rangle$ of t_3 for which $c' \neq !c$, returns to the starting marking.

Example 4 In the small SN shown in Fig. 8, we first encounter arc functions whose images are multisets with multiplicities greater than one. This raises no theoretical issues for our calculus; instead, it is forbidden in [14]. All possible semiflows can be checked directly. For example, $\mathbf{I}_p^1$ states that for p_1 , the sum of the colors in the two components of its 2-tuples always matches the multiset of colors in p_2 .

6 Generating A Semi-Flow Basis

The capability to formally verify symbolic semiflows is particularly valuable, as modelers typically possess a priori knowledge regarding the expected system behavior and seek to corroborate this knowledge through automated verification support. Nevertheless, the capability to construct a semiflow generative family represents an even more critical issue from both theoretical and practical standpoints. As mentioned previously, existing theoretical approaches have achieved only partial success, as they rely on imposing rather stringent syntactic constraints on the admissible classes of arc functions.

In this section, we concisely present a hybrid empirical–theoretical methodology that, instead of relying on advanced algebraic frameworks, exploits Property 4.1 under the standing assumption that all arc functions have a constant size.

Claim 6.1. *The set of arc functions in $\mathcal{L}_{\mathcal{D}_\Sigma, \mathcal{D}'_\Sigma}$ of a certain size k is finite and can be computed explicitly.*

The preceding claim is naturally justified by the fact that the collection of class-functions with domain $\mathcal{D}_\Sigma$ is uniquely fixed, and the symbolic color-tuples in the domain $\mathcal{D}'_\Sigma$ of cardinality k can be generated by an algorithmic enumeration. In general, this enumeration problem, which is theoretically nontrivial, may become computationally expensive when the parameter k is large and-or the color domains are composite. From this point of view, Property 4.2 may help.

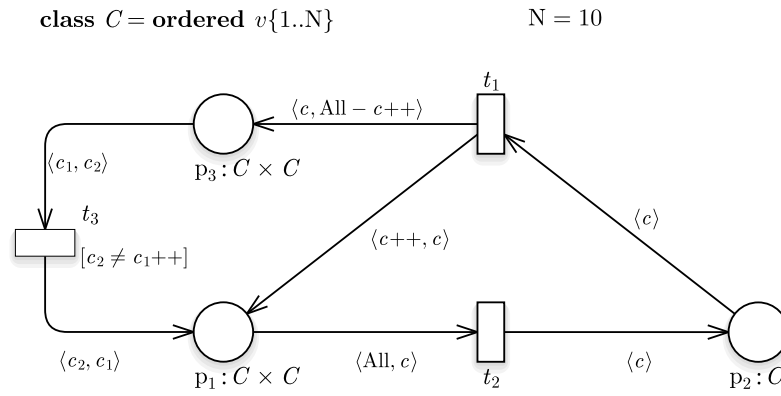


Figure 6: An SN with an ordered class (++ is !)

$$\begin{array}{c}
 \begin{array}{c} p_1 \\ p_2 \\ p_3 \end{array} \begin{bmatrix} \begin{array}{c} t_{1C} \\ t_{2C} \\ t_{3C,C} \end{array} \\ \begin{array}{c} \langle !c, c \rangle \\ -\langle All, c \rangle \\ \langle c_2, c_1 \rangle [c_2 \neq !c_1] \end{array} \\ \begin{array}{c} \langle c, All - !c \rangle \\ -\langle c \rangle \\ -\langle c_1, c_2 \rangle [c_2 \neq !c_1] \end{array} \end{bmatrix}
 \end{array}$$

$$\begin{array}{c}
 \begin{array}{c} \mathbf{I}_P^1 \\ \mathbf{I}_P^2 \end{array} \begin{array}{c} p_1 \\ p_2 \\ p_3 \end{array} \begin{bmatrix} \begin{array}{c} \langle c_2 \rangle_{C,C} \\ \langle c_1 \rangle_{C,C} \end{array} \\ \begin{array}{c} N \langle c \rangle_C \\ \langle All \rangle \\ \langle c_2 \rangle_{C,C} \end{array} \end{bmatrix} \\
 \mathbf{I}_T \begin{array}{c} t_1 \\ t_2 \\ t_3 \end{array} \begin{bmatrix} \langle c \rangle_C \\ \langle c \rangle_C \\ \langle c, All - !c \rangle_C \end{bmatrix}
 \end{array}$$

Figure 7: Incidence Matrix and semiflows of SN in Fig. 6

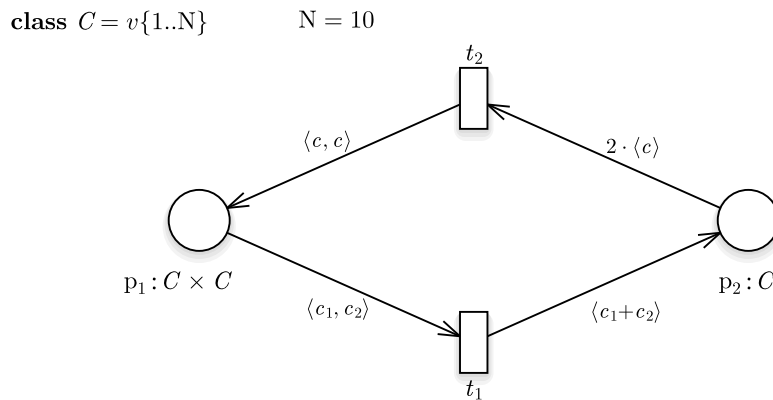


Figure 8: An SN with functions with non-one coefficients

$$\begin{array}{c}
\begin{array}{cc}
& t_{1C,C} & t_{2C} \\
p_1 & \left[-\langle c_1, c_2 \rangle_{C,C} & \langle c, c \rangle_C \right] \\
p_2 & \left[\langle c_1 + c_2 \rangle_{C,C} & -2\langle c \rangle_C \right]
\end{array} \\
\begin{array}{cc}
p_1 & p_2 \\
\mathbf{I}_P & \left[\langle c_1 + c_2 \rangle_{C,C} & \langle c \rangle_C \right]
\end{array} \\
\begin{array}{cc}
t_1 & t_2 \\
\mathbf{I}_T & \left[\langle c, c \rangle_C & \langle c \rangle_C \right]
\end{array}
\end{array}$$

Figure 9: Incidence Matrix and semiflows of SN in Fig. 8

We argue that the difficulty of this task can be significantly reduced by leveraging certain properties of SN arc functions. In particular, any arc function F can be expressed in the form $\sum_i \lambda_i T_i$, with $\lambda_i \in \mathbb{N}$, where each T_i is a tuple (optionally preceded by a filter) that produces multisets in which all element multiplicities are equal to one.

In this paper, we do not introduce any algorithm; instead, we demonstrate how to use the previous claim to construct a semiflow generative family, drawing on a simple example from Section 5. More challenging cases arise when considering the other examples in that section.

Consider the SN depicted in Fig. 2, whose underlying skeleton is provided in the Appendix (Fig. 14). The P-semiflow basis of this skeleton consists of the two vectors $[\text{start} : 1 \quad \text{line} : 1 \quad \text{ass} : 1]$ and $[\text{ok} : 1 \quad \text{focc} : 1]$. In order to derive the corresponding symbolic P-semiflows, we must first select a common subdomain for the places belonging to the support of each semiflow. In this case, there is one candidate subdomain for each of the two P-semiflows, namely L and PL , respectively. These classes are neither ordered nor partitioned; consequently, the corresponding admissible class-functions are $\{l, All_L\}$ and $\{pl, All_{PL}\}$. Depending on the place under consideration, these functions may have different domains (for instance, l_L for the place *start* and $l_{PL,L}$ for the place *work*); we let the context resolve this ambiguity. Hence, by concentrating on the first P-semiflow and observing that the only arc functions of cardinality one that can be constructed are $\langle l \rangle$ if $K > 2$ and either $\langle l \rangle$ or $\langle All - l \rangle$ if $K = 2$, we directly conclude that the symbolic P-semiflow $\mathbf{I}_p^1$ depicted in Fig. 3 constitutes part of the basis for $K > 2$. If $K = 2$, the P-vector $[\text{start} : \langle All - l \rangle_L \quad \text{line} : \langle All - l \rangle_{PL,L} \quad \text{ass} : \langle All - l \rangle_L]$ also constitutes a semiflow, which is equivalent to the previously defined one. We do not elaborate further on this aspect here. The basis of P-semiflows is completed by $\mathbf{I}_p^2$ (Fig. 3), for which analogous comments can be made.

The unique minimal T-semiflow of the SN skeleton is given by

$$[\text{load} : 1 \quad \text{work} : K \quad \text{assembly} : 1],$$

And therefore, the only admissible subdomain of transition domains is PL . This subdomain will serve as the common domain for all functions that make up the symbolic T-semiflow. With respect to the transition *work*, we must determine arc functions of cardinality $K := |L|$. The only feasible candidates for the corresponding semiflow component are $\langle pl, All_L \rangle$ and, in the specific case where $N(= |PL|) = K = 2$, the additional candidate $2\langle All - pl, All_L \rangle$.

7 Colored Flows

Symbolic flows capture notable properties, both qualitative (the types of colors) and quantitative (the number of colors). In general, however, they comply with rather restrictive, conservative patterns.

It is frequently advantageous to verify qualitative and quantitative invariants separately. In particular, the preservation of specific structural properties of colored place markings can be independent of the precise cardinality of tokens present, for example in the case of unbounded models. This is exactly the purpose of *colored semiflows*, hereafter referred to simply as Csemiflows.

Let $\mathbf{H}^+$ and $\mathbf{H}^-$ be the $|P| \times |T|$ matrices defined entrywise by

$$\mathbf{H}^+[p, t] = \overline{O(p, t) \ominus I(p, t)}, \quad \mathbf{H}^-[p, t] = \overline{I(p, t) \ominus O(p, t)}.$$

Equivalently, for a given color instance of transition t , the entries $\mathbf{H}^+[p, t]$ and $\mathbf{H}^-[p, t]$ represent, respectively, the sets of color tuples (tokens) that are added and removed from place p .

Definition 7.1 (P-Csemiflow). *Let $\mathbf{I}_{P-c}$ be an $1 \times |P|$ vector of functions such that $\mathbf{I}_{P-c}[p] : \mathcal{D}(p) \rightarrow 2^{\mathcal{D}'}$ and $\mathbf{I}_{P-c}[p] \neq 0 \Rightarrow \mathcal{D}' \leq \mathcal{D}(p)$.*

$\mathbf{I}_{P-c}$ is a P-Csemiflow if and only if $\mathbf{I}_{P-c} \cdot \mathbf{H}^+ \equiv \mathbf{I}_{P-c} \cdot \mathbf{H}^-$.

T-Csemiflows are defined analogously.

Example 5 Consider the SN depicted in Fig. 10. The underlying skeleton admits one P-semiflow: $[p_1 : 2 \quad p_2 : 1 \quad p_3 : 3]$; It is straightforward to verify that there are no symbolic semiflows that match it (using property 4.1 and considering the functions of proper size). Each transition is equipped with a guard, and there is also a self-loop between t_1 and p_2 . A transition guard is naturally propagated over the incident arc functions. To construct the matrices $\mathbf{H}^+$ and $\mathbf{H}^-$, we must rewrite the difference $\mathbf{H}[p_2, t_1] := O(p_2, t_1) - I(p_2, t_1)$ as a pairwise disjoint algebraic sum, which can always be achieved in our framework. In this simple situation, we obtain (omitting the tuple notation; $g_1 \equiv \Phi(t_1)$):

$$(-2c_1 + c_2)[g_1] \equiv c_2[c_1 \neq c_2][g_1] - (c_1[c_1 == c_2] + 2c_1[c_1 \neq c_2])[g_1]$$

When considering the support of positive and negative terms, we remove multiplicities and obtain $\mathbf{H}^+[p_2, t_1] = c_2[c_1 \neq c_2 \wedge g_1]$, $\mathbf{H}^-[p_2, t_1] = c_1[g_1]$. (Hereinafter, in this subsection, we omit the multiset support notation for simplicity.). The other elements of $\mathbf{H}^+$ and $\mathbf{H}^-$ are simply derived.

At this stage, we can verify (by applying the calculus in $\overline{\mathcal{L}}$) whether the two vectors $\mathbf{I}_P^i$ depicted in Fig. 11 actually define colored semiflows. The first would, in that case, correspond to the conservation of the colors in place p_2 and the first component of the 2-tuples in place p_1 , respectively, with respect to the subclass C_1 . However, this verification fails. Consider, for example, the product of $\mathbf{I}_P^1$ with the first column of $\mathbf{H}^+$ and, analogously, with the first column of $\mathbf{H}^-$:

$$\begin{aligned} \langle c \rangle [c \in C_1] \circ \langle c_2 \rangle [g_1 \wedge c_1 \neq c_2] &\equiv \langle c_2 \rangle [g_1 \wedge c_1 \neq c_2] \\ \langle c_2 \rangle [c_2 \in C_1] \circ \langle c_1, c_2 \rangle [g_1] + \langle c \rangle [c \in C_1] \circ \langle c_1 \rangle [g_1] &\equiv \langle c_2 \rangle [g_1] + \langle c_1 \rangle [g_1] \end{aligned}$$

the resulting expressions, of course, are not equivalent. In contrast, we easily verify that $\mathbf{I}_P^2$ is a Csemiflow: the set of colors in place p_3 and the first component of the 2-tuples in p_1 remain invariant.

An interesting concluding observation is that the calculus in $\overline{\mathcal{L}}$ implemented in SNexpression is inherently parametric with respect to the cardinalities of color classes, which can be specified by linear constraints; for example, $|C| \geq 3$. Consequently, we are able to validate Csemiflows not only in a purely symbolic manner but also in a parametric setting. For instance, the aforementioned results hold irrespective of the cardinality of the subclass C_1 . A detailed investigation of this parametric analysis is beyond the scope of the present work.

class $C = v\{1..3\}$ is $C_1 + v\{4..N\}$ is C_2 $N = 10$

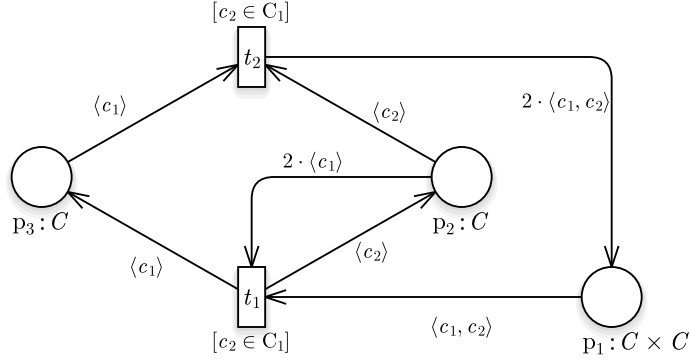


Figure 10: SN with a split color class and guards

$$\begin{array}{c}
 \mathbf{H}^+ \quad \begin{array}{cc} t_{1,C,C} & t_{2,C,C} \\ \begin{array}{l} p_1 \\ p_2 \\ p_3 \end{array} \left[\begin{array}{c} \langle c_2 \rangle [g_1 \wedge c_1 \neq c_2] \\ \langle c_1 \rangle [g_1] \end{array} \right] \end{array} & \mathbf{H}^- \quad \begin{array}{cc} t_{1,C,C} & t_{2,C,C} \\ \begin{array}{l} p_1 \\ p_2 \end{array} \left[\begin{array}{c} \langle c_1, c_2 \rangle [g_1] \\ \langle c_2 \rangle [g_2] \\ \langle c_1 \rangle [g_2] \end{array} \right] \end{array} \\
 \\
 \mathbf{I}_P^1 \quad \begin{array}{ccc} p_1 & p_2 & p_3 \\ \left[\begin{array}{c} \langle c_2 \rangle [c_2 \in C_1]_{C,C} \\ \langle c_1 \rangle_{C,C} \end{array} \right] & \langle c \rangle [c \in C_1]_C & \langle c \rangle_C \end{array} & \mathbf{I}_P^2 \quad \begin{array}{ccc} p_1 & p_2 & p_3 \\ \left[\begin{array}{c} \langle c_2 \rangle [c_2 \in C_1]_{C,C} \\ \langle c_1 \rangle_{C,C} \end{array} \right] & \langle c \rangle [c \in C_1]_C & \langle c \rangle_C \end{array}
 \end{array}$$

Figure 11: $\mathbf{H}^+$ and $\mathbf{H}^-$ matrices and P-Csemiflow of SN in Fig. 10

8 Verifying More General Invariants

Using a simple yet nontrivial example, we then briefly illustrate how the SN structural calculus implemented in SNexpression can be employed to rigorously verify properties that go beyond symbolic or colored semiflows. With this method, we broaden the class of models that can be examined (including, for instance, unbounded models) and account for inhibitor arcs, which substantially enhance the expressive power of the SN modeling framework.

The approach is based on a well-established principle. Our goal is to establish property $\mathcal{P}$, which is formally defined in an extended language $\mathcal{L}^+$ of $\mathcal{L}$ (or $\overline{\mathcal{L}}^+$ if we disregard the quantitative aspects).

1. We first show that $\mathcal{P}$ is satisfied in the initial marking (which can also be described symbolically).
2. We then show that if $\mathcal{P}$ is met in a generic marking $\mathbf{m}$, then it is also met in any $\mathbf{m}'$ such that $\mathbf{m}[(t, c) > \mathbf{m}', \text{ for all } t \in T, c \in \mathcal{D}(t)]$.

Our focus is on the second point, which is the most significant. The essential requirement is that this proof be carried out at a symbolic level, that is, by operating on symbolic transition instances that compactly encode the corresponding concrete instances. To this end, and in analogy with SMT-based

class $C = v\{1..N\}$ $N = 10$

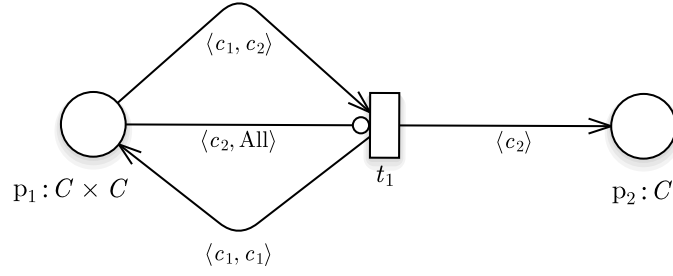


Figure 12: An SN transition with inhibitor edge

techniques, we employ the SNexpression rewriting engine to infer new axioms (rewriting rules), which are then dynamically incorporated into the underlying theory. Currently, this inference process is performed manually. However, we plan to extend the SNexpression CLI to make the overall workflow semi-automated.

From a technical point of view, we must enrich $\mathcal{L}$ ($\overline{\mathcal{L}}$) with symbols that denote parametric (multi)sets over specified color domains. We use symbols like $Z_{\mathcal{D}_\Sigma}$ to denote a *parametric* (multi)set in $\mathcal{D}_\Sigma$, where $\mathcal{D}_\Sigma$ denotes any subdomain made up of color classes in Σ . This symbol can be interpreted as a function with a null (or neutral) domain; that is, $\bullet \rightarrow \text{Bag}[\mathcal{D}_\Sigma]$ or $\bullet \rightarrow 2^{\mathcal{D}_\Sigma}$, depending on whether we are considering $\mathcal{L}^+$ or $\overline{\mathcal{L}}^+$.

The functional operators of the calculus are then applied in a uniform manner. For example, we may form expressions such as $\langle Z_{\mathcal{D}_\Sigma}^1, Z_{\mathcal{D}_\Sigma}^2 \rangle$, $Z_{\mathcal{D}_\Sigma}^1 \mu Z_{\mathcal{D}_\Sigma}^2$, or $F \circ Z_{\mathcal{D}_\Sigma}^1$, where $\mu \in \{+(-), \cap, \ominus\}$, $F \in \{\mathcal{D}_\Sigma \rightarrow D\}$.

Example 6 Let us clarify the concept by means of the SN in Fig. 12. Our goal is to show that the first component of the color 2-tuples in place p_1 and the colors in place p_2 are disjoint, under the assumption that this holds in the initial marking $\mathbf{m}_0$. Demonstrating this directly by inspecting the color functions is not straightforward at all. We can represent an arbitrary SN marking $\mathbf{m}$ as: $[\mathbf{m}(p_1) : Z_{C,C}^1 \quad \mathbf{m}(p_2) : Z_C^1]$. The invariant property is (we suppose that ‘ $\circ$ ’ takes priority over ‘ $\cap$ ’ and the latter over ‘ $+$ ’):

$$c_1 \circ Z_{C,C}^1 \cap Z_C^1 \equiv 0$$

The first natural step consists of incorporating the corresponding rewriting rule $c_1 \circ Z_{C,C}^1 \cap Z_C^1 \longrightarrow 0$ into the framework. Subsequently, we syntactically encode within $\mathbf{m}$ the enabling condition associated with transition t_1 . This procedure can, in principle, be fully automated. A generic symbolic instance of t_1 is given by $(t_1, \langle Z_C^2, Z_C^3 \rangle)$, where Z_C^2 and Z_C^3 denote parametric colors in C that are bound to variables (projections) c_1 and c_2 , respectively.

The constraints imposed by the input and inhibitor arc functions can be represented in the form of rewrite rules. With respect to the input constraint, we obtain:

$$Z_{C,C}^1 \longrightarrow Z_{C,C}^2 + \langle Z_C^2, Z_C^3 \rangle, \quad |Z_C^2| = 1, |Z_C^3| = 1.$$

By substituting this encoding into the invariant expression, which is supposed to hold in $\mathbf{m}$, and then applying the composition rules together with standard multiset properties—such as the distributivity of composition over multiset sum and the following

$$(A + B) \cap C \equiv 0 \Rightarrow A \cap C \equiv 0 \wedge B \cap C \equiv 0,$$

we finally obtain this expression, which can be simplified:

$$c_1 \circ (Z_{C,C}^2 + \langle Z_C^2, Z_C^3 \rangle) \cap Z_C^1 \equiv c_1 \circ Z_{C,C}^2 \cap Z_C^1 + Z_C^2 \cap Z_C^1 \equiv 0$$

From which we infer the rules: i) $c_1 \circ Z_{C,C}^2 \cap Z_C^1 \longrightarrow 0$, ii) $Z_C^2 \cap Z_C^1 \longrightarrow 0$.

The constraint imposed by the inhibitor arc function is formalized as follows:

$$\begin{aligned} \langle Z_C^3, All \rangle \cap (Z_{C,C}^2 + \langle Z_C^2, Z_C^3 \rangle) &< \langle All, All \rangle \cong \\ \langle Z_C^3, All \rangle \cap (Z_{C,C}^2 + \langle Z_C^2, Z_C^3 \rangle) &\equiv 0 \cong \\ \langle Z_C^3, All \rangle \cap Z_{C,C}^2 &\equiv 0 \wedge \langle Z_C^3, All \rangle \cap \langle Z_C^2, Z_C^3 \rangle \equiv 0 \end{aligned}$$

From which we infer the rules: iii) $Z_C^3 \cap c_1 \circ Z_{C,C}^2 \longrightarrow 0$, iv) $Z_C^3 \cap Z_C^2 \longrightarrow 0$.

The marking expression we obtain by symbolically firing the instance $(t_1, \langle Z_C^2, Z_C^3 \rangle)$ in $\mathbf{m}$ is:

$$[\mathbf{m}'(p_1) : Z_{C,C}^2 + \langle Z_C^2, Z_C^2 \rangle \quad \mathbf{m}'(p_2) : Z_C^1 + Z_C^3].$$

Finally, it remains to establish the following result:

$$\begin{aligned} c_1 \circ (Z_{C,C}^2 + \langle Z_C^2, Z_C^2 \rangle) \cap (Z_C^1 + Z_C^3) &\equiv (c_1 \circ Z_{C,C}^2 + Z_C^2) \cap (Z_C^1 + Z_C^3) \equiv 0 \cong \\ c_1 \circ (Z_{C,C}^2) \cap Z_C^1 &\equiv 0 \wedge c_1 \circ (Z_{C,C}^2) \cap Z_C^3 \equiv 0 \wedge Z_C^2 \cap Z_C^1 \equiv 0 \wedge Z_C^2 \cap Z_C^3 \equiv 0 \end{aligned}$$

which can be directly inferred from the axioms i)-iv).

As a final remark, we note that this methodology can also be employed to validate candidate symbolic P-semiflows (or colored flows). However, in this case, the validation is achieved through a more involved and less streamlined procedure than the approaches presented in Sections 5 and 7.

9 Conclusions

We have demonstrated that the symbolic structural calculus for Symmetric Nets (SN), developed over the past two decades to derive structural dependencies between SN nodes and implemented in the SNexpression tool, can also be employed to validate a broad class of symbolic (semi)flows for extended Symmetric Net (ESN) models. Furthermore, we have undertaken a preliminary investigation into the derivation of a semiflow basis by exploiting a natural correspondence with the conventional semiflows of the ESN skeleton. Finally, we have outlined how the calculus can be used to formally verify more general classes of structural invariants.

Practically, we plan to extend SNexpression by (1) fully supporting composition of multiset functions, (2) adding an interactive inference mechanism that feeds newly derived axioms into the SNexpression rewriting engine, and (3) enabling smooth interoperability with other tools, especially GreatSPN. Theoretically, we study whether generalized inverses of arc functions (extended linearly) can be expressed symbolically to construct a generative family of symbolic flows, potentially building on this paper's preliminary results.

References

- [1] Elvio Gilberto Amparore, Gianfranco Balbo, Marco Beccuti, Susanna Donatelli & Giuliana Franceschinis (2016): *30 years of GreatSPN*. In: *Principles of Performance and Reliability Modeling and Evaluation*, Springer, pp. 227–254, doi:10.1007/978-3-319-30599-8_9.
- [2] Matteo Camilli & Lorenzo Capra (2021): *Formal specification and verification of decentralized self-adaptive systems using symmetric nets*. *Discrete Event Dynamic Systems*, doi:10.1007/s10626-021-00343-3.
- [3] L. Capra, M. De Pierro & G. Franceschinis (2005): *A High Level Language for Structural Relations in Stochastic Well-formed Nets*. In: *Applications and Theory of Petri Nets 2005*, 3536, Springer-Verlag, pp. 168–187, doi:10.1007/11494744_11.
- [4] L. Capra, M. De Pierro & G. Franceschinis (2015): *Computing Structural Properties of Symmetric Nets*. In: *Quantitative Evaluation of Systems, Proceedings of the 12th International Conference, QEST 2015*, 9259, Springer International Publishing, pp. 125–140, doi:10.1007/978-3-319-22264-6_9.
- [5] L. Capra, M. De Pierro & G. Franceschinis (2020): *SNexpression: A Symbolic Calculator for Symmetric Net Expressions*. In: *Proc. of the 41st Int. Conference Petri Nets 2020*, LNCS 12152, Springer, pp. 381–391, doi:10.1007/978-3-030-51831-8_19.
- [6] L. Capra, M. De Pierro & G. Franceschinis (2021): *General Composition for Symmetric Net Arc Functions with Applications*. In: *Proceedings of the Int. Workshop on Petri Nets and Software Engineering 2021, CEUR workshop proceedings 2907*.
- [7] L. Capra, M. De Pierro & G. Franceschinis (2025): *SNexpression: A New Component for SN Matrix-Based Structural Analysis*. In: *Formal Techniques for Distributed Objects, Components, and Systems: 45th IFIP WG 6.1 International Conference, FORTE 2025, Held as Part of the 20th International Federated Conference on Distributed Computing Techniques, DisCoTec 2025, Lille, France, June 16–20, 2025, Proceedings*, Springer-Verlag, Berlin, Heidelberg, p. 193–201, doi:10.1007/978-3-031-95497-9_12.
- [8] L. Capra & M. Köhler-Bußmeier (2024): *Modular rewritable Petri nets: An efficient model for dynamic distributed systems*. *Theoretical Computer Science* 990, p. 114397, doi:10.1016/j.tcs.2024.114397.
- [9] Lorenzo Capra (2024): *Modular Stochastic Rewritable Petri Nets*. In Pedro Cabalar, Francesco Fabiano, Martin Gebser, Gopal Gupta & Theresa Swift, editors: *Proceedings 40th International Conference on Logic Programming, ICLP 2024, University of Texas at Dallas, Dallas Texas, USA, October 14-17 2024, EPTCS 416*, pp. 128–134, doi:10.4204/EPTCS.416.11.
- [10] G. Chiola, C. Dutheillet, G. Franceschinis & S. Haddad (1993): *Stochastic well-formed colored nets and symmetric modeling applications*. *IEEE Transactions on Computers* 42(11), pp. 1343–1360, doi:10.1109/12.247838.
- [11] José Manuel Colom & Manuel Silva (1989): *Convex geometry and semiflows in P/T nets. A comparative study of algorithms for computation of minimal P-semiflows*. In: *International Conference on Application and Theory of Petri Nets*, Springer, pp. 79–112, doi:10.1007/3-540-53863-1_22.
- [12] J. M. Couvreur, S. Haddad & J. F. Peyre (1993): *Generative families of positive invariants in Coloured nets sub-classes*. In Grzegorz Rozenberg, editor: *Advances in Petri Nets 1993*, Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 51–70, doi:10.1007/3-540-56689-9_39.
- [13] J. M. Couvreur & J. Martínez (1991): *Linear Invariants in Commutative High Level Nets*, pp. 284–302. Springer Berlin Heidelberg, Berlin, Heidelberg, doi:10.1007/978-3-642-84524-6_9. Available at https://doi.org/10.1007/978-3-642-84524-6_9.
- [14] S. Evangelista, C. Pajault & J. F. Pradat-Peyre (2007): *A Simple Positive Flows Computation Algorithm for a Large Subclass of Colored Nets*. In John Derrick & Jüri Vain, editors: *Formal Techniques for Networked and Distributed Systems – FORTE 2007*, Springer Berlin Heidelberg, pp. 177–195, doi:10.1007/978-3-540-73196-2.

- [15] K. Jensen (1991): *Coloured Petri Nets: A High Level Language for System Design and Analysis*, pp. 44–119. Springer Berlin Heidelberg, Berlin, Heidelberg, doi:10.1007/978-3-642-84524-6_2. Available at https://doi.org/10.1007/978-3-642-84524-6_2.
- [16] K. Jensen & L. M. Kristensen (2009): *Coloured Petri Nets: Modelling and Validation of Concurrent Systems*. Springer, Netherlands, doi:10.1007/b95112.
- [17] K. Jensen & G. Rozenberg, editors (1991): *High-level Petri Nets: Theory and Application*. Springer-Verlag, London, UK, doi:10.1007/978-3-642-84524-6.
- [18] W. Reisig (1985): *Petri Nets: An Introduction*. Springer-Verlag New York, Inc., New York, NY, USA, doi:10.1007/978-3-642-69968-9.
- [19] M. Silva, J. Martinez, P. Ladet & H. Alla (1991): *Generalized Inverses and the Calculation of Symbolic Invariants for Coloured Petri Nets*, pp. 303–315. Springer Berlin Heidelberg, Berlin, Heidelberg, doi:10.1007/978-3-642-84524-6_10.

start_l1	line_pl1_l2	line_pl2_l2	ass_pl1_l2	ass_pl2_l2
start_l1	line_pl1_l1	line_pl2_l2	ass_pl1_l1	ass_pl2_l2
start_l1	line_pl1_l2	line_pl2_l1	ass_pl1_l2	ass_pl2_l1
start_l1	line_pl1_l1	line_pl2_l1	ass_pl1_l1	ass_pl2_l1
start_l2	line_pl1_l2	line_pl2_l2	ass_pl1_l2	ass_pl2_l2
start_l2	line_pl1_l1	line_pl2_l2	ass_pl1_l1	ass_pl2_l2
start_l2	line_pl1_l2	line_pl2_l1	ass_pl1_l2	ass_pl2_l1
start_l2	line_pl1_l1	line_pl2_l1	ass_pl1_l1	ass_pl2_l1

load_p_pl2	work_l_l1_p_pl2	work_l_l2_p_pl2	assembly_p_pl2
load_p_pl1	work_l_l1_p_pl1	work_l_l2_p_pl1	assembly_p_pl1

Table 1: P- and T-semiflows of the Petri net shown in Fig. 13.

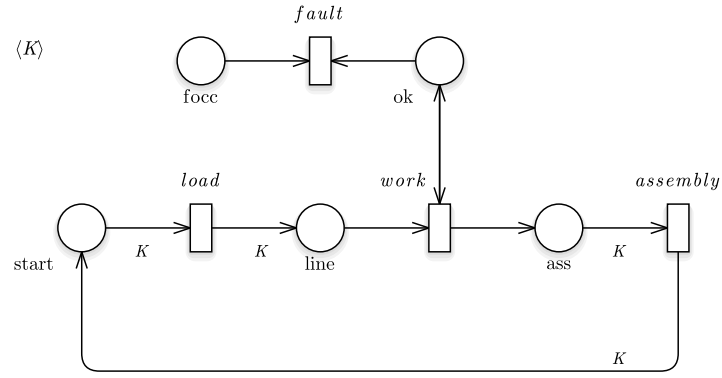


Figure 14: Skeleton of the PL model

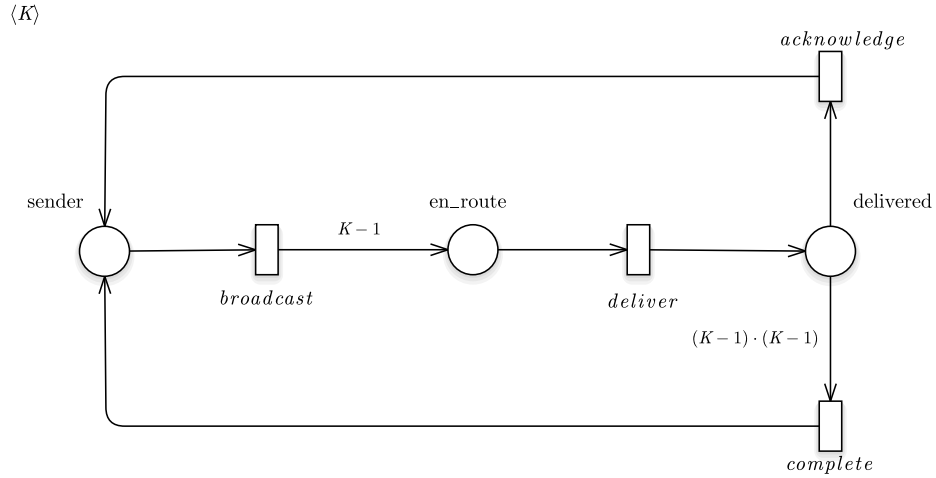


Figure 15: Skeleton of the broadcast model